

Chapter 3: Control Flow

Relational graph edition. Pull-shaped. Top-down.

How to use this document

Red = checkpoint (answer in chat). Green = practice. Blue = target. Walk edges in order.

Big picture

Control flow is the routing table of your program. Every program is a sequence of decisions (if/switch) and repetitions (while/for/do). This chapter gives you exactly 6 constructs. Picking the right one for each situation is the skill.

Target problem

Write a simple integer calculator that reads lines of the form operand operator operand (e.g., 42 + 17), dispatches on the operator using switch, loops until EOF, and handles bad input with else-if chains.

The Graph

Node inventory

Node	Syntax	When to use
if-else	if (expr) stmt else stmt	Two-way decision
else-if	if ... else if ... else	Multi-way decision with complex conditions
switch	switch (expr) { case: ... }	Multi-way on a single integer/char value
while	while (expr) stmt	Loop with test at top, may execute 0 times
for	for (init; test; incr) stmt	Counted/indexed loops (most common loop)
do-while	do stmt while (expr);	Loop with test at bottom, always executes once
break	break;	Exit innermost loop or switch immediately
continue	continue;	Skip to next iteration of innermost loop
goto	goto label;	Jump anywhere in function (almost never needed)

Decision graph -- which construct to use

Q: How many branches?

2 branches → if-else

3+ branches on same int/char value → switch

3+ branches on different conditions → else-if chain

Q: How many iterations?

Known count / index variable → for

Unknown count, may be zero → while

Must execute at least once → do-while

Zoom in -- one edge at a time

Edge 1: if-else

The expression is evaluated. If nonzero (true), the first statement executes. If zero (false), the else branch executes (if present). **Dangling else trap:** an else binds to the nearest preceding if without an else.

```
if (n > 0)
    result = positive;
else
    result = non_positive;
```

Problem 3.1: Write an if-else that sets sign to -1, 0, or 1 depending on whether x is negative, zero, or positive.

Answer: `if (x < 0) sign = -1; else if (x == 0) sign = 0; else sign = 1;`

Edge 2: switch

Tests a single integer/char expression against constant cases. Execution falls through from one case to the next unless you use break. default catches everything not matched.

```
switch (operator) {
    case '+': result = a + b; break;
    case '-': result = a - b; break;
    case '*': result = a * b; break;
    case '/': result = (b != 0) ? a/b : 0; break;
    default: printf("bad op\n"); break;
}
```

Fall-through is intentional in C. Forgetting break is a common bug. Intentional fall-through should have a comment.

Problem 3.2: Write a switch that counts vowels and consonants in input. Use fall-through to group vowel cases.

Answer: `case 'a': case 'e': case 'i': case 'o': case 'u': vowels++; break;` default handles consonants.

Edge 3: while loop

Test at the top. If the condition is false initially, the body never executes. Use when you do not know how many iterations in advance.

```
int c;
while ((c = getchar()) != EOF) {
    /* process c */
}
```

Problem 3.3: Use a while loop to reverse the digits of an integer n. (Hint: extract last digit with $n\%10$, remove it with $n/10$.)

Answer: `int rev=0; while(n>0){rev=rev*10+n%10; n/=10;}`

Edge 4: for loop

The most common loop. All loop machinery (init, test, increment) on one line. Preferred for counted loops and array traversals. Any of the three parts can be omitted (infinite loop: `for(;;)`).

```
for (int i = 0; i < n; i++)  
    printf("%d\n", a[i]);
```

Problem 3.4: Write a for loop that computes n factorial.

Answer: `long fact=1; for(int i=2; i<=n; i++) fact*=i;`

CHECKPOINT A

Without looking up: when do you use switch vs else-if? When for vs while? What happens if you forget break in a switch case?

Edge 5: do-while

Body executes at least once, then tests. Rare in practice -- most common use is digit-by-digit number processing (you always have at least one digit, even for 0).

```
/* Convert int n to string s */
int i = 0;
do {
    s[i++] = n % 10 + '0';
} while ((n /= 10) > 0);
s[i] = '\0';
```

Problem 3.5: Why would a while loop fail for the itoa conversion above when n is 0?

Answer: while(n>0) would never enter the body for n=0, producing an empty string.
do-while always produces at least '0'.

Edge 6: break and continue

break exits the innermost enclosing loop or switch. continue skips the rest of the current iteration and jumps to the next test (while/do) or increment (for). Neither affects outer loops.

Problem 3.6: Write a loop that reads ints from input and sums only the positive ones. Use continue to skip negatives. Stop on 0.

Answer: while(scanf("%d",&n)==1){if(n==0)break; if(n<0)continue;
sum+=n;}

Edge 7: goto (rarely needed)

Only legitimate use: breaking out of deeply nested loops. In all other cases, break/continue/return/restructuring is clearer. If you use goto, something is probably wrong with your design.

Solving the target problem

Walk: E3 (while for input loop) + E2 (switch for dispatch) + E1 (else-if for error handling)

```
#include <stdio.h>

int main(void) {
    int a, b;
    char op;
    while (scanf("%d %c %d", &a, &op, &b) == 3) {
        switch (op) {
            case '+': printf("%d\n", a + b); break;
            case '-': printf("%d\n", a - b); break;
            case '*': printf("%d\n", a * b); break;
            case '/':
                if (b == 0)
                    printf("error: divide by zero\n");
                else
                    printf("%d\n", a / b);
                break;
            default:
                printf("unknown operator: %c\n", op);
                break;
        }
    }
    return 0;
}
```

CHECKPOINT B

Add support for the % (modulus) operator to the calculator. Handle the divide-by-zero case the same way. Report your modified switch.

Final exam

Problem 1: Write a function `void itoa(int n, char s[])` that converts integer `n` to string. Handle negative numbers.

Problem 2: Write a function `void reverse(char s[])` that reverses string `s` in place.

Problem 3: Write a binary search function using a while loop (not for).

Problem 4: Rewrite the calculator to also handle parenthesized single operations like `(3 + 4)`.

Problem 5: Write a function `void shellsort(int v[], int n)` using nested for loops with a gap sequence.

CHECKPOINT C

Solve at least 3 of the 5 problems. Report in chat.

Answer key

1. itoa: Walk E5 (do-while for digit extraction) + E1 (if for negative). Handle sign, reverse result string.
2. reverse: Walk E4 (for loop, two indices moving inward). Swap $s[i]$ and $s[j]$ until they meet.
3. binary search: Walk E3 (while $lo \leq hi$). Compute mid, compare, adjust lo or hi.
4. Read '(' then operand operator operand then ')'. Use getchar for parens, scanf for the rest.
5. shellsort: Walk E4 (outer for with gap $\neq 2$, inner for with insertion comparison). Gap starts at $n/2$.